

D206: Data Cleaning

Hannah Bergman

WGU Email: hbergm3@wgu.edu

Table of Contents

Part I: Research Question.....	1
A: Research Question.....	2
B: Variables in the Data Set & Data Types.....	2
Part II: Data-Cleaning Plan.....	15
C: Data-Cleaning Plan.....	16
C1. Relevant Techniques & Specific Steps To Assess Data Quality.....	16
C2. Approach for Assessing Data Quality.....	16
C3. Programming Language, Libraries & Packages.....	17
C4. Annotated Code (Executable Script File).....	17
Part III: Data Cleaning.....	17
D: Data Cleaning Process Summary.....	17
D1. Findings for Data Quality Issues.....	17
D2. Methods for Mitigating Data Quality Issues.....	18
D3. Outcome From Implementation of Each Data-Cleaning Step.....	22
D4. Annotated Code (Executable Script File).....	22
D5. Cleaned Data Set (External CSV File).....	22
D6. Limitations of the Data-Cleaning Process.....	22
D7. How These Limitations Could Affect Analysis.....	23
E: Principal Component Analysis (PCA).....	24
E1. Total Number of Principal Components & Output of PC Loading Matrix.....	24
E2. Justification of Reduced Number of Principal Components.....	26
E3. How Our Organization Would Benefit From PCA.....	27
Part IV: Supporting Documents.....	28
F: Video Demonstration.....	28
G: Data Source.....	28
H: References.....	28

Part I: Research Question

A: Research Question

What patient factors are influenced by a patient being overweight?

Defining The Research Question With Our Data

- **Overweight:**

Our data captures overweight/obesity status as a categorical boolean variable rather than more specific body weight. While slightly limited, this factor is important for our hospital to consider due to the global public health burden of obesity, particularly in Westernized nations like the United States.

- **Patient factors:**

Our data goes beyond just body weight to capture a broader spectrum of patient health. We track the presence of conditions like **stroke**, **diabetes**, **back pain**, **asthma**, **hyperlipidemia** (high cholesterol) and **high blood pressure**, recognizing the interconnectedness of these factors in shaping overall well-being. Focusing on these comorbidities is particularly important because:

- According to the Centers for Disease Control and Prevention (2022), “obesity-related conditions include heart disease, **stroke**, type 2 **diabetes** and certain types of cancer”.
- Obesity is a known contributing factor to both the development and aggravation of **back pain**. The additional weight can exert undue pressure on the spinal structures, leading to musculoskeletal imbalances and pain (Whelan, 2023).
- In regards to lung health, “the prevalence of **asthma** is increased in obesity, particularly in adult women” (Dixon & Nyenhuis, 2023).
- Being overweight or obese elevates the risk of developing metabolic syndrome, characterized by risk factors like high cholesterol [**hyperlipidemia**], insulin resistance, and **high blood pressure**. These factors, in turn, increase the likelihood of cardiovascular disease and type 2 diabetes (John Hopkins Medicine, n.d.).

Relevancy To Our Organization

Building on existing medical research, which suggests a correlation between obesity and increased prevalence of other health factors, this study aims to investigate whether this association holds true within our own patient population. We hypothesize that overweight patients admitted to our hospital will exhibit a higher incidence of comorbidities commonly linked to obesity, providing targeted evidence to support or refute this broader hypothesis.

B: Variables in the Data Set & Data Types

Patient Information

The dataset features data on:

- **Readmissions (ReAdmis):** Identifies patients who were readmitted to the hospital within one month of discharge.
- **Medical Factors:** Includes specific documented conditions like stroke, diabetes, back pain, asthma, hyperlipidemia (high cholesterol), and high blood pressure, as well as the level of complication risk for the patient should they need more extensive treatment.
- **Hospitalization Information:** Details the service received, length of stay, type of initial admission, and charges associated with hospitalization and treatment.
- **Patient Demographics:** Captures patient information such as location, gender, age, marital status, number of children, employment status, job title, income and education level.

Data Overview

- **Number of patients/rows:** 10,000
- **Number of variables/columns:** 52

Variable 1 name: CaseOrder

Data Type: Quantitative (int)

Description: This placeholder variable represents the row index, a unique serialized integer assigned to each row in the dataset to preserve the original order of the raw data file.

Observable value from the dataset:

CaseOrder
1

Variable 2 name: Customer_id

Data Type: Qualitative (str)

Description: This variable assigns a unique ID to each patient, allowing for individual tracking and analysis.

Observable value from the dataset:

Customer_id
L298599

Variable 3 name: Interaction

Data Type: Qualitative (str)

Description: Like Customer_id, this unique code allows for the association of individual patient interactions with their corresponding records, including patient transactions, procedures and admissions.

Observable value from the dataset:

Interaction
875e42a5-cff2-46e6-add9-3d7e841c2e99

Variable 4 name: UID

Data Type: Qualitative (str)

Description: Like Customer_id and Interaction, this unique identifier (UID) allows for the association of individual patient UIDs with their corresponding records, including patient transactions, procedures and admissions..

Observable value from the dataset:

UID
ca5c80e63f580807e127770cc9bc4db8

Variable 5 name: City

Data Type: Qualitative (str)

Description: This attribute identifies the patient's self-reported city of residence from the billing statement to categorize their geographic location at the city level.

Observable value from the dataset :

City
Great Falls

Variable 6 name: State

Data Type: Qualitative (str)

Description: This attribute identifies the patient's self-reported state of residence from the billing statement to categorize their geographic location at the state level.

Observable value from the dataset:

State
MT

Variable 7 name: County

Data Type: Qualitative (str)

Description: This attribute identifies the patient's self-reported county of residence from the billing statement to categorize their geographic location at the county level.

Observable value from the dataset:

County

Cascade

Variable 8 name: Zip

Data Type: Qualitative (str)

Description: This attribute identifies the patient's geographic location through their self-reported zip code on the billing statement

Observable value from the dataset:

Zip
59404

Variable 9 name: Lat

Data Type: Quantitative (float)

Description: This attribute identifies the patient's geographic location by GPS coordinate, observing the latitude (Lat) through the residence listed on the billing statement.

Observable value from the dataset:

Lat
47.6306

Variable 10 name: Lng

Data Type: Quantitative (float)

Description: This attribute identifies the patient's geographic location by GPS coordinate, observing the longitude (Lng) through the residence listed on the billing statement.

Observable value from the dataset:

Lng
-111.3471

Variable 11 name: Population

Data Type: Quantitative (int)

Description: This field records the population within a mile radius of the patient, based on census data.

Observable value from the dataset:

Population
27618

Variable 12 name: Area

Data Type: Qualitative (str)

Description: This attribute categorizes the patient's location area type as Urban, Suburban, or Rural, based on unofficial census data.

Observable value from the dataset:

Area
Urban

Variable 13 name: Timezone**Data Type:** Qualitative (str)**Description:** This field indicates the time zone associated with the patient's address as listed during registration.**Observable value from the dataset:**

Timezone
America/Denver

Variable 14 name: Job**Data Type:** Qualitative (str)**Description:** This attribute captures the professional title of the patient, as listed in their admission information, or the occupation of the primary insurance holder if different.**Observable value from the dataset:**

Job
Data scientist

Notes: It's difficult to determine from the data whether the job title belongs to the patient, a caregiver, spouse, or parent. This ambiguity could lead to misinterpretation.**Variable 15 name:** Children**Data Type:** Quantitative (int)**Description:** This attribute lists the number of children in the patient's household as reported in their admissions information.**Observable value from the dataset:**

Children
0

Variable 16 name: Age**Data Type:** Quantitative (int)**Description:** This field identifies the self-reported age of the patient at the time of admission, as documented in their admissions information.**Observable value from the dataset:**

Age
31

Variable 17 name: Education

Data Type: Qualitative (str)

Description: This field identifies the highest earned degree of the patient, according to their admissions information as No Schooling Completed, Nursery School to 8th Grade, 9th Grade to 12th Grade (No Diploma), GED or Alternative Credential, Regular High School Diploma, Professional School Degree, Some College (Less than 1 Year), Some College (1 or More Years - No Degree), Associate's Degree, Bachelor's Degree, Master's Degree, Doctorate Degree.

Observable value from the dataset:

Education
Master's Degree

Variable 18 name: Employment

Data Type: Qualitative (str)

Description: This attribute categorizes the employment status of the patient as reported in their admissions information as Unemployed, Part Time, Full Time, Retired or Student.

Observable value from the dataset:

Employment
Full Time

Variable 19 name: Income

Data Type: Quantitative (float)

Description: This field records the annual income of the patient, as reported during time of admission, or the income of the primary insurance holder if different.

Observable value from the dataset:

Income
103866.49

Notes: As with Job title, it's unclear from the data whether the income belongs to the patient, a caregiver, spouse, or parent. To avoid misinterpretation, I believe the income data requires clarification regarding ownership. Unreported income from inflated patient bills could burden patients with unexpected costs and raise ethical questions about transparency and fair billing practices. Discrepancies between what patients are charged and what our hospital reports as income could lead to financial strain for patients, potential losses for our organization due to audits, and may erode trust due to ethical concerns.

Variable 20 name: Marital

Data Type: Qualitative (str)

Description: This field categorizes the marital status of the patient, as listed in their admission information, or the marital status of the primary insurance holder if different. Categories include: Never Married, Married, Separated, Divorced, Widowed.

Observable value from the dataset:

Martial
Married

Notes: Again, this field is unclear as to whether the marital status refers to the patient or the insurance policyholder.

Variable 21 name: Gender

Data Type: Qualitative (str)

Description: This field identifies the self-identified gender of the patient as Male, Female, or Prefer not to answer (Nonbinary).

Observable value from the dataset:

Gender
Prefer not to answer

Notes: The current data dictionary conflates “Prefer not to answer” with “Nonbinary”, potentially limiting patient self-identification and hindering access to appropriate care. We’re capturing gender identity, which in our modern American society is considered a fluid social construct that is distinct from biological sex assigned at birth. Having only 3 limited options might not accurately reflect patients’ diverse identities. This has the potential to significantly impact care. For example, a transgender male patient who identifies as masculine (“Male”) might miss crucial gynecological care due to a mismatch in recorded identity and actual needs. A broader spectrum of options, possibly including Male, Female, Transgender Male (assigned Female at birth), Transgender Female (assigned Male at birth), Genderfluid, Gender Neutral, Nonbinary, Prefer not to say, and Other (with a text box for self-identification), would empower patients to express their identities more accurately and help ensure they receive appropriate healthcare tailored to their specific needs.

Variable 22 name: ReAdmis

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether a patient was readmitted to the hospital within a month of their previous discharge.

Observable value from the dataset:

ReAdmis
Yes

Variable 23 name: VitD_levels

Data Type: Quantitative (float)

Description: This attribute records the patient’s Vitamin D levels, measured in nanograms per milliliter (ng/mL).

Observable value from the dataset:

VitD_levels
17.80233049

Variable 24 name: Doc_visits

Data Type: Quantitative (int)

Description: This column indicates how many times the patient saw their primary physician during their initial hospitalization.

Observable value from the dataset:

Doc_visits
6

Variable 25 name: Full_meals_eaten

Data Type: Quantitative (int)

Description: This column details the number of complete meals each patient consumed while hospitalized. Only full meals are counted, and any partial meals are excluded (entered as zero). Some patients were permitted, upon request, to receive more than the standard three meals per day.

Observable value from the dataset:

Full_meals_eaten
2

Variable 26 name: VitD_supp

Data Type: Quantitative (int)

Description: This field indicates the number of times that Vitamin D supplements were administered to the patient.

Observable value from the dataset:

VitD_supp
2

Variable 27 name: Soft_drink

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value ("Yes" or "No") indicating whether a patient habitually drinks 3 or more soft drinks in a day.

Observable value from the dataset:

Soft_drink
No

Variable 28 name: Initial_admin

Data Type: Qualitative (str)

Description: This field classifies the patient's initial admission method as Emergency, requiring immediate medical attention; Elective, scheduled in advance; or Observation, for monitoring without immediate treatment.

Observable value from the dataset:

Initial_admin
Emergency Admission

Variable 29 name: HighBlood

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value ("Yes" or "No") indicating whether a patient has High Blood Pressure.

Observable value from the dataset:

HighBlood
Yes

Variable 30 name: Stroke

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value ("Yes" or "No") indicating whether a patient has had a Stroke.

Observable value from the dataset:

Stroke
No

Variable 31 name: Complication_risk

Data Type: Qualitative (str)

Description: This column reflects the patient's potential for complications, categorized as High, Medium, or Low, following a comprehensive initial assessment.

Observable value from the dataset:

Complication_risk
High

Variable 32 name: Overweight

Data Type: Qualitative (boolean)

Description: This column represents a boolean value (1 = True = "Yes" or 0 = False = "No") indicating whether the patient is considered overweight based on the patient's age, gender and height.

Observable value from the dataset:

Overweight
1

Variable 33 name: Arthritis

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has Arthritis

Observable value from the dataset:

Arthritis
Yes

Variable 34 name: Diabetes

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has Diabetes.

Observable value from the dataset:

Diabetes
Yes

Variable 35 name: Hyperlipidemia

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has Hyperlipidemia (more colloquially known as high cholesterol).

Observable value from the dataset:

Hyperlipidemia
No

Variable 36 name: BackPain

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has Back Pain.

Observable value from the dataset:

BackPain
No

Variable 37 name: Anxiety

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has an Anxiety disorder.

Observable value from the dataset:

Anxiety
No

Variable 38 name: Allergic_rhinitis

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has Allergic Rhinitis.

Observable value from the dataset:

Allergic_rhinitis
Yes

Variable 39 name: Reflux_esophagitis

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has Reflux Esophagitis.

Observable value from the dataset:

Reflux_esophagitis
No

Variable 40 name: Asthma

Data Type: Qualitative (boolean)

Description: This column represents a text representation of a boolean value (“Yes” or “No”) indicating whether the patient has Asthma.

Observable value from the dataset:

Asthma
Yes

Variable 41 name: Services

Data Type: Qualitative (str)

Description: This attribute indicates the primary Service the patient received while hospitalized, such as Blood Work, Intravenous, CT Scan, or MRI.

Observable value from the dataset:

Services
CT Scan

Variable 42 name: Initial_days

Data Type: Quantitative (float)

Description: This column specifies the number of days the patient stayed in the hospital during their initial visit.

Observable value from the dataset:

Initial_days
4.772177208

Variable 43 name: TotalCharge

Data Type: Quantitative (float)

Description: This column displays the average daily charge for patients. It's calculated by dividing the total charges by the number of days spent in the hospital. This amount represents the typical costs patients are billed in USD, excluding specialized treatments.

Observable value from the dataset:

TotalCharge
2774.08992

Notes: The “TotalCharge” label of this column is blatantly misleading. It implies that the data represents the **SUM** of all charges incurred by the patient during their entire stay, however according to the data dictionary, it actually reflects an **AVERAGE** daily charge, excluding some treatments. To ensure clarity and avoid misinterpretation, this column should be renamed to something more accurate to the data being recorded, such as “AvgDaily_charges”.

Variable 44 name: Additional_charges

Data Type: Quantitative (float)

Description: This column displays the average amount patients are charged for additional healthcare services in USD, including procedures, treatments, medications, and anesthesiology.

Observable value from the dataset:

Additional_charges
12742.58991

Notes: Like “TotalCharge”, this label is very misleading. “Additional_charges” implies that the data represents the **SUM** of all additional charges incurred by the patient during their stay, while the data dictionary defines this as the **AVERAGE** amount charged to the patient for those miscellaneous things. To ensure clarity and avoid misinterpretation, this column should be renamed to something more accurate to the data being recorded, such as “AvgAdditional_charges”.

Variable 45 name: Item1

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of timely admission. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item1
1

Variable 46 name: Item2

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of timely treatment. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item2
2

Variable 47 name: Item3

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of timely visits. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item3
3

Variable 48 name: Item4

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of reliability. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item4
4

Variable 49 name: Item5

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of options. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item5
5

Variable 50 name: Item6

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of hours of treatment. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item6
6

Variable 51 name: Item7

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of courteous staff. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item7
7

Variable 52 name: Item8

Data Type: Qualitative (int)

Description: This column captures patients' qualitative responses to a survey question about the importance of evidence of active listening from the doctor. While the data is stored as numerical values (1-8), it represents a textual rating scale, where 1 indicates "most important" and 8 indicates "least important".

Observable value from the dataset:

Item8
8

Part II: Data-Cleaning Plan

C: Data-Cleaning Plan

C1. Relevant Techniques & Specific Steps To Assess Data Quality

Step 1. Review the organization's Data Dictionary (D206 Data Cleaning_Medical Data Considerations and Dictionary.pdf) to ensure understanding of the variables, their data formats, and associated descriptions.

Step 2. Use the `info()` method within pandas to provide a comprehensive high-level overview of the DataFrame, including:

- Column & entry count: Verifies the expected 52 columns and 10,000 entries (rows), as per the data dictionary.
- Column labels: Lists each column's name.
- Non-Null Count: Reveals the number of non-empty cells in each column. This is crucial for identifying missing values, represented as "NaN", which should ideally match the total row count (10000 in this case) for complete data.
- Dtype (Data Types): Displays the assigned data type for each column. We can check this against the data dictionary to ensure the data type is appropriate for each column.

Step 3. Begin going through each column individually to examine the data within, checking for duplicates, missing values and outliers.

- Identify duplicates with the `duplicated()` method. If there are any duplicates, the output will return 'True' on the correlating row.
- With `df.isnull().sum()`, we can check how many values are missing by showing the sum of the missing values (values that are NaN, None, or NaT) in each column.
- Check how many unique values are in our columns with the `nunique()` method. This method is helpful for checking our unique and serialized columns, like CaseOrder, Customer_id and UID, which have counts of 10,000 since there are 10,000 rows, each having its own unique value. It also helps ensure accuracy with our categorical variables. For example, our State column should only have a count of 50 since there are only 50 states in the US.
- Similarly, with the `counts()` method, we can verify categorical data that should have a specified category according to our data dictionary, such as Area which should have counts for the categories of Rural, Suburban, and Urban.
- Using the `describe()` method and the functions `nsmallest()` and `nlargest()`, we can see if our numerical data exists within reasonable bounds to identify any potential outliers. We can also use Plotly to visualize this information to make it easier to spot where the outliers fall within our data.

C2. Approach for Assessing Data Quality

This plan takes a two-pronged approach to data cleaning:

- **Big picture view:** We'll analyze the entire dataset to identify overall patterns and potential issues.
- **Column-level inspection:** We'll then zoom in on each column to evaluate the data quality within it.

To ensure our approach aligns with the intended use of the data, we'll reference the provided data dictionary. This way, our cleaning decisions consider the business significance of each variable. Overall, this approach combines a comprehensive overview with meticulous detail, leading to a thorough and consistent data cleaning process.

C3. Programming Language, Libraries & Packages

For this project, we'll be using **Python**, which provides simplicity and high readability. This language also offers an extensive range of libraries for several tasks related to this project, including:

- **Pandas:** provides a spreadsheet-like structure called a DataFrame, allowing us to store and manipulate the CSV data more efficiently
- **Matplotlib:** enhances our exploratory data analysis by allowing us to easily visualize our data to find inconsistencies
- **Plotly:** allows us to create more dynamic plots to visualize our data more in-depth

Once the data is clean, we'll perform Principal Component Analysis (PCA) using:

- **Scikit-learn's PCA:** to analyze the data's principal components
- **Numpy:** for our mathematical operations, helping us transform the data
- **Matplotlib:** to create an informative scree plot to visualize the PCA results

C4. Annotated Code (Executable Script File)

See code attached via file 'd206_task1_c4.ipynb'

Part III: Data Cleaning

D: Data Cleaning Process Summary

D1. Findings for Data Quality Issues

- There are several columns that do not have the correct counts of 10,000, indicating missing or undefined NaN values within our records:
 - **"Children"** (count: 7,412, indicating 2,588 missing values)
 - **"Age"** (count: 7,586, indicating 2,414 missing values)
 - **"Income"** (count: 7,536, indicating 2,464 missing values)

- **“Soft_drink”** (count: 7,533, indicating 2,467 missing values)
 - **“Overweight”** (count: 9,018, indicating 982 missing values)
 - **“Anxiety”** (count: 9,016, indicating 984 missing values)
 - **“Initial_days”** (count: 8,944, indicating 1,056 missing values)
- Several columns require data type review due to potential inconsistencies
- Several columns contain an unexpected number of unique entries:
 - **“State”**: Given the presence of locations beyond the 50 US states, "State" doesn't fully encompass the data accurately.
 - **“Timezone”**: Discrepancy exists between the 26 listed time zones and the 7 recognized US time zones.
- Several column labels require revision due to potential misinterpretation:
 - **“TotalCharge”**: Despite implying the sum of all patient charges, this column represents the **average daily charge** excluding specific treatments (as defined in the data dictionary).
 - **“Additional_charges”**: While suggesting the sum of all additional charges, it represents the **average amount charged** for miscellaneous items (as defined in the data dictionary).
- Some columns are unclear in their labeling:
 - **“Income”** and **“Job”**: The current data lacks clarity on whether the recorded income and job information pertains to the patient themselves or someone else involved in their care, such as a caregiver, spouse, or parent. This ambiguity can lead to potential billing and data analysis errors.
 - **“Gender”**: To ensure optimal patient care and respect for diverse identities, the existing "Gender" field might require re-evaluation. Consider expanding options or offering open-ended responses to capture a wider range of identifiers.
 - **“Item#”**: Currently, the "Item#" column alone is ambiguous and uninformative without referencing the separate survey data it presumably links to. Providing context within the column label or including a descriptive explanation would improve clarity and usability.
- There are multiple naming conventions used for labeling:
 - Currently, inconsistencies exist (camel-case, underscores, mixed), which can hinder readability and comprehension. Aligning with Python's naming convention (lowercase, separated by underscores) would promote clarity and consistency, especially considering our Python-based workflow.

D2. Methods for Mitigating Data Quality Issues

- Remove duplicate values:
 - Use `drop_duplicates()` method to drop any duplicates from the dataset. It doesn't appear that there are any from what we explored in C4 but as a precaution we'll still do this in case any values marked 'True' were overlooked.
- Fix missing data:
 - **“Children”**: Pregnancy and childbirth do have potential long-term health concerns, however, according to the data dictionary this column refers to the

number of children living in a patient's household, **not** biological children. With that understanding, we'll impute the NaN values to 0 using the `fillna(0)` method

- **"Age"**: While examining the "Age" data, it becomes obvious that there are no patients younger than 18. However, simply filling missing ages with values >18 wouldn't reflect the diverse health concerns across different age groups. Imputing the missing values with the average age (53) might seem reasonable, but the wide age range (18-89) highlights significant health differences between someone in their 50s and someone in their 80s. Therefore, we'll opt to impute missing ages with 0 using the `fillna(0)` method. This avoids introducing inaccurate information by assuming all missing ages are 18, and having a value of 0 clearly flags missing data prompting further investigation.
- **"Initial_days"**: This column currently uses a float data type, but according to our data dictionary, charges are billed daily. It can be reasonably assumed that the NaN values indicate 0 day stays given the minimum possible stay and the absence of values below 1.00000000. Based on this assumption, it's also reasonable to conclude that this column tracks days and hours. With that understanding, we'll use the `fillna(0)` method to impute 'NaN' values to 0 to account for those patients whose stays were less than a full day.
- **"Soft_drink", "Overweight", "Anxiety"**: Due to potential health concerns related to obesity, anxiety and soft drink consumption, simply replacing NaN values in these columns with 0 could be misleading and potentially detrimental to the health of patients. Therefore, we'll use the `fillna('')` method to replace all of these 'NaN' values with blank spaces, showing the records as blank. This approach cleans up our data while ensuring transparency and facilitating further investigation into the missing data.
- **"Income"**: Since there are no values for income under 154.08, it's reasonable to assume that NaN records mean the policyholder records no income. Therefore, we'll opt to impute missing ages with 0 using the `fillna(0)` method. We may want to consider using `fillna('')` instead since not reporting income may impact a patient's financial assistance eligibility, however, doing this automatically reassigns this variable from a float to an object, which is not ideal since income is very obviously a quantitative variable and should be represented by a floating point number or integer.
- **"Item#"**: Use `fillna(0)` method to impute 'NaN' values to 0. We'll later remap that value to "4: Neutral" (so as not to skew survey results) in a new internal dictionary, `standardized_survey_responses`, which will map survey responses with descriptive explanations of values for improved clarity and usability.
- Change the data types of columns with the `astype()` method:
 - **"Overweight" and "Anxiety"**: These columns store data in floating-point format, which is unnecessarily resource-intensive. Since they only contain simple "0" and "1" values, converting them to the Boolean (bool) datatype would significantly

improve storage efficiency. We'll convert these columns to the bool data type using `df["column_name"].astype("bool")`.

- **“ReAdmis”, “Soft_drink”, “HighBlood”, “Stroke”, “Arthritis”, “Hyperlipidemia”, “BackPain”, “Allergic_rhinitis”, “Reflux_esophagitis”, “Diabetes” and “Asthma”**: These string objects only contain simple “Yes” and “No” values so converting them to the Boolean (bool) datatype would also significantly improve storage efficiency. We'll convert all these columns to the bool data type using `astype("bool")`.
- **“Area”, “Timezone”, “Education”, “Employment”, “Marital”, “Gender”, “Inital_admin”, “Complication_risk”, “Services”, “Item#”**: Would be better stored as category data types. These columns represent predefined categories, making them ideal for this change. This migration will improve data integrity, optimize data processing, and potentially reduce storage consumption. We'll convert all these columns to the bool data type using `astype("category")`.
- **“Zip”**: Storing zip codes as integers can truncate leading zeros, which are crucial for precise location identification. Using a string data type would prevent this data loss and ensure accuracy. We'll convert this column to the string data type using `astype("str")`. Additionally, we can use the `zfill(5)` method to set the length of our zip codes to 5 characters long (since US zip codes are 5 digits).
- **“Children”**: The current data type for "Children" is a float, which seems counterintuitive as having fractional children isn't possible. Converting this column to an integer would align better with the real-world meaning and prevent potential data analysis errors. We'll use `fillna(0)` to impute NaN values with 0 and then cast the column to the int64 data type with `astype("int64")`.
- **“Age”**: While currently a float, storing "Age" as an integer aligns better with its data (no decimals) and improves efficiency. We'll use `fillna(0)` to impute NaN values with 0 and then cast the column to the int64 data type with `astype("int64")`.
- **“Inital_days”**: Converting this column to an integer better reflects the data and its intended use. Additionally, rounding each value to the nearest single-digit integer and replacing NaN values with 0 (representing the minimum possible stay) would ensure consistency and prevent data analysis issues. We'll use `fillna(0)` to impute NaN values with 0 and then cast the column to the int64 data type with `astype("int64")`.
- **Replace and reformat data:**
 - **“VitD_Levels”, “TotalCharge” & “Additional_charges”**: Use the `round(2)` function to display float values with 2 decimal places instead of 8 for improved readability.
 - **“Timezone”**: Standardize all entries to match the recognized timezones for enhanced data accuracy and ease of interpretation. This will involve some research to determine an equivalent for each timezone and adding onto the data dictionary the new timezone categories, as well as using the `replace()` method to replace values with the new time zones.

- **“Gender”** Use the `replace()` method to replace ‘Prefer not to answer’ with ‘Non-Binary’, as defined in the data dictionary. Redefining the option will greatly improve the meaning of this data.
- **“Item#”** Use the `replace()` method to recategorize survey responses to their equivocal meanings for more apparent understanding. Rather than an entry simply being ‘1’ with no other context, it will become ‘1: Most Important’.
- Relabel columns:
 - Create a new variable called `updated_labels` to store a new array of updated column names that are standardized and more accurate to the data they reflect:

Standardize label names (lowercase, separated by underscores):

- **“CaseOrder”** becomes **“case_order”**
- **“Customer_id”** becomes **“customer_id”**
- **“Interaction”** becomes **“interaction_id”**
- **“UID”** becomes **“uid”**
- **“City”** becomes **“city”**
- **“County”** becomes **“county”**
- **“Population”** becomes **“population”**
- **“Age”** becomes **“age”**
- **“Education”** becomes **“education”**
- **“VitD_levels”** becomes **“vitamin_d_level”**
- **“VitD_supp”** becomes **“vitamin_d_supplements”**
- **“Stroke”** becomes **“stroke”**
- **“Overweight”** becomes **“overweight”**
- **“Arthritis”** becomes **“arthritis”**
- **“Diabetes”** becomes **“diabetes”***
- **“Hyperlipidemia”** becomes **“hyperlipidemia”**
- **“BackPain”** becomes **“back_pain”**
- **“Anxiety”** becomes **“anxiety”**
- **“Allergic_rhinitis”** becomes **“allergic_rhinitis”**
- **“Reflux_esophagitis”** becomes **“reflux_esophagitis”**
- **“Asthma”** becomes **“asthma”**
- **“Services”** becomes **“services_provided”**

Redefine labels to make them easier to understand:

- **“State”** becomes **“region”** (*accounts for states, territories, districts, etc*)
- **“Zip”** becomes **“zip_code”**
- **“Lat”** becomes **“latitude_gps_coordinate”**
- **“Lng”** becomes **“longitude_gps_coordinate”**
- **“Area”** becomes **“area_type”**
- **“Timezone”** becomes **“time_zone”**
- **“Employment”** becomes **“employment_status”**
- **“Gender”** becomes **“gender_identity”**
- **“Children”** becomes **“children_in_household”**
- **“ReAdmis”** becomes **“is_readmission”**
- **“Doc_visits”** becomes **“physician_visits”**

- **"Full_meals_eaten"** becomes **"Full_meals_consumed"**
- **"Soft_drink"** becomes **"habitual_soda_consumption"**
- **"Initial_admin"** becomes **"initial_admission_type"**
- **"HighBlood"** becomes **"high_blood_pressure"**
- **"Complication_risk"** becomes **"complication_risk_level"**
- **"Initial_days"** becomes **"initial_days_hospitalized"**

Clarify this data belongs to the insurance policyholder, not necessarily the patient:

- **"Income"** becomes **"policyholder_income"**
- **"Marital"** becomes **"policyholder_marital_status"**
- **"Job"** becomes **"policyholder_job"**

Correct the labels to the data they *actually* reflect:

- **"TotalCharge"** becomes **"avg_daily_charges"**
- **"Additional_charges"** becomes **"avg_additional_charges"**

Make the labels easier to understand without outside resources:

- **"Item1"** becomes **"survey_question1_timely_admission"**
- **"Item2"** becomes **"survey_question2_timely_treatment"**
- **"Item3"** becomes **"survey_question3_timely_visits"**
- **"Item4"** becomes **"survey_question4_reliability"**
- **"Item4"** becomes **"survey_question5_options"**
- **"Item6"** becomes **"survey_question6_hours_of_treatment"**
- **"Item7"** becomes **"survey_question7_courteous_staff"**
- **"Item8"** becomes **"survey_question8_physician_actively_listens"**

D3. Outcome From Implementation of Each Data-Cleaning Step

The above implementations will help simplify future analysis of this data set, as well improve data accuracy and integrity when the data is updated. Converting columns to boolean and categories helps standardize data entry, restricting confusing input and ensuring consistency. Reformatting floating-point numbers (vitamin D levels, charges) enhances readability. Using integers for single-digit numbers (age, children, hospital days) saves storage space for better efficiency and clarifies the meaning of the data. Relabeling columns with clearer names more accurate to the data they represent reduces reliance on external references like the data dictionary and survey dataset and promotes intuitive analysis of this data set. Renaming "Item#" columns and labeling responses with descriptions in addition to plain numerical values with no context provides better clarity to survey responses and meanings. Additionally, more explicit labels like "Non-binary" for "Prefer not to answer" in "Gender" could positively impact patient care.

D4. Annotated Code (Executable Script File)

See code attached via file 'd206_task1_d4_e1_e2.ipynb'

D5. Cleaned Data Set (External CSV File)

See CSV file attached via file 'medical_clean_data.csv'

D6. Limitations of the Data-Cleaning Process

Despite the above data cleaning methods, significant ambiguity and attribution issues still remain within our patient data:

- Uncertainties around whether income, job, and marital status pertain to the patient or a connected individual (spouse, parent, caregiver) create misinterpretation risks, the biggest of which being financial burden. Undisclosed or unrepresented income could leave patients facing unexpected costs or inflated bills, which in turn raises ethical concerns about our organization's transparency and fair billing practices.
- The current data dictionary conflates "Prefer not to answer" with "Nonbinary", potentially limiting patient self-identification and hindering patient access to appropriate care. We're capturing gender identity, which in our modern American society is considered a fluid social construct that is distinct from biological sex assigned at birth. Having only 3 limited options might not accurately reflect patients' diverse identities. This has the potential to significantly impact care. For example, a transgender male patient who identifies as masculine ("Male") might miss crucial gynecological care due to a mismatch in recorded identity and actual needs. A broader spectrum of options, possibly including Male, Female, Transgender Male (assigned Female at birth), Transgender Female (assigned Male at birth), Genderfluid, Gender Neutral, and Nonbinary, would empower patients to better express their identities more accurately and help ensure they receive appropriate healthcare tailored to their specific needs.
- "Diabetes" encompasses several distinct types, each exhibiting varying degrees of correlation with other comorbidities. According to the Centers for Disease Control and Prevention (2023), Type 2 Diabetes is often preventable and potentially self-induced, stemming from excess weight and unhealthy dietary habits. In contrast, Type 1 Diabetes is less prevalent and arises from a genetic autoimmune response, independent of diet and lifestyle choices. Additionally, Gestational Diabetes warrants separate consideration due to its unique characteristics associated with pregnancy. Simply grouping all types of diabetes into a single column is quite restrictive, particularly for my research question, which aims to explore potential correlations between obesity and specific health conditions, such as Type 2 Diabetes.
- Similarly, simply labeling a patient as 'Overweight' is far too broad and lacks crucial nuance. "Overweight" could refer to the clinical range (BMI 25-29.9) or be loosely used for anyone above an ideal, healthy weight. This ambiguity creates confusion, even as someone with a limited medical background. The health risks and implications of being 2 pounds overweight are **vastly different** from someone carrying 200+ pounds of excess weight. Lumping them together undermines the severity of one and minimizes the potential dangers of the other.
- The rounding of some of our floating point columns should be reviewed to ensure the data is suitable for use outside of our analysis.
- I have serious concerns about the accuracy and transparency of specific data points, particularly "TotalCharge" and "Additional_charges". "TotalCharge" implies the **sum** of all charges but the data dictionary reveals it's actually an **average daily charge**, excluding treatments. "Additional_charges" similarly suggests the **total** of additional charges but

the data dictionary clarifies it's the **average cost** for miscellaneous items. While I've corrected these labels via data cleaning, these discrepancies are potentially deceptive in why they exist in the first place.

D7. How These Limitations Could Affect Analysis

Misinterpretation & Bias:

- **Attribution issues:** Uncertainties about who data pertains to (patient vs. policyholder) create major misinterpretation risk, potentially leading to inaccurate analysis.
- **Limited gender options:** Confusing “Non-binary” with “Prefer not to answer” plus only having 3 limited gender options limits patient self-identification and access to appropriate care, skewing data about demographics.
- **Oversimplified diagnoses:** Grouping health conditions like “Diabetes” into a single oversimplified singular diagnosis ignores crucial distinctions between different types and correlating health concerns. Analyses might miss crucial distinctions between different types of health conditions, leading to inaccurate conclusions.
- **Overweight label ambiguity:** Lumping patients under a single “Overweight” label ignores the vast health difference between being slightly overweight and carrying significant excess weight, obscuring risk factors and hindering research into questions like my own.

Confusing & Misleading Data:

- **Mislabeled charges:** The changes made to the labels “TotalCharge” and “Additional_charges” now accurately reflect the data but could exclude some expenses, leading to inaccurate financial analysis.

Overall Impact:

- **Biased & incomplete conclusions:** These limitations can lead to biased and incomplete conclusions in research and analysis, jeopardizing the effectiveness of healthcare decisions and impacting patient care negatively.
- **Ethical concerns:** Misleading data raises ethical concerns about transparency, fair billing practices, and potential discrimination based on inaccurate information.

E: Principal Component Analysis (PCA)

See code attached via file ‘d206_task1_d4_e1_e2.ipynb’

E1. Total Number of Principal Components & Output of PC Loading Matrix

Principal Component Analysis (PCA) was employed on available quantitative variables in the dataset to identify underlying patterns. These variables included geographical coordinates (latitude_gps_coordinate & longitude_gps_coordinate), demographic characteristics (population, children_in_household, age, policyholder_income), health metrics (vitamin_d_level, physican_visits), dietary habits (full_meals_consumed, vitamin_d_supplements), and hospitalization details (initial_days_hospitalized, avg_daily_charges, avg_additional_charges).

Screenshot:

E1. Principal Component Analysis (PCA)

Purpose: Identify the total number of principal components and provide the output of the principal components loading matrix

```
# STORE QUANTITATIVE VARIABLES

# Create a dedicated data frame containing only quantitative variables for PCA (principal component analysis)
df_pca = df[["latitude_gps_coordinate", "longitude_gps_coordinate", "population", "children_in_household", "age", "p

# NORMALIZE DATA

# Standardize each variable by subtracting the mean and dividing by the std (standard deviation)
df_normalized_pca = (df_pca - df_pca.mean()) / df_pca.std()

# SET PCA SIZE

# Specify the number of principal components (PCs) to retain
pca = PCA(n_components=df_pca.shape[1])
```

```
# FIT DATA

# Fit the normalized data to the PCA model, capturing most important directions of variance in the data
pca.fit(df_normalized_pca)
```

```
PCA
PCA(n_components=13)
```

```
# CREATE PRINTABLE PCA DATA

# Create a data frame containing the transformed data projected onto the principal components
printable_pca = pd.DataFrame(pca.transform(df_normalized_pca), columns=["PC1", "PC2", "PC3", "PC4", "PC5", "PC6", "P

# GENERATE LOADINGS

# Create a data frame showing the loadings for each PC
# Each loading represents the correlation between a variable and a PC, indicating how much each feature contributes
pca_loadings = pd.DataFrame(pca.components_.T, columns = ["PC1", "PC2", "PC3", "PC4", "PC5", "PC6", "PC7", "PC8", "P

# PRINT LOADINGS

# Print the data frame to examine the correlations and interpret the meaning of each PC
pca_loadings
```

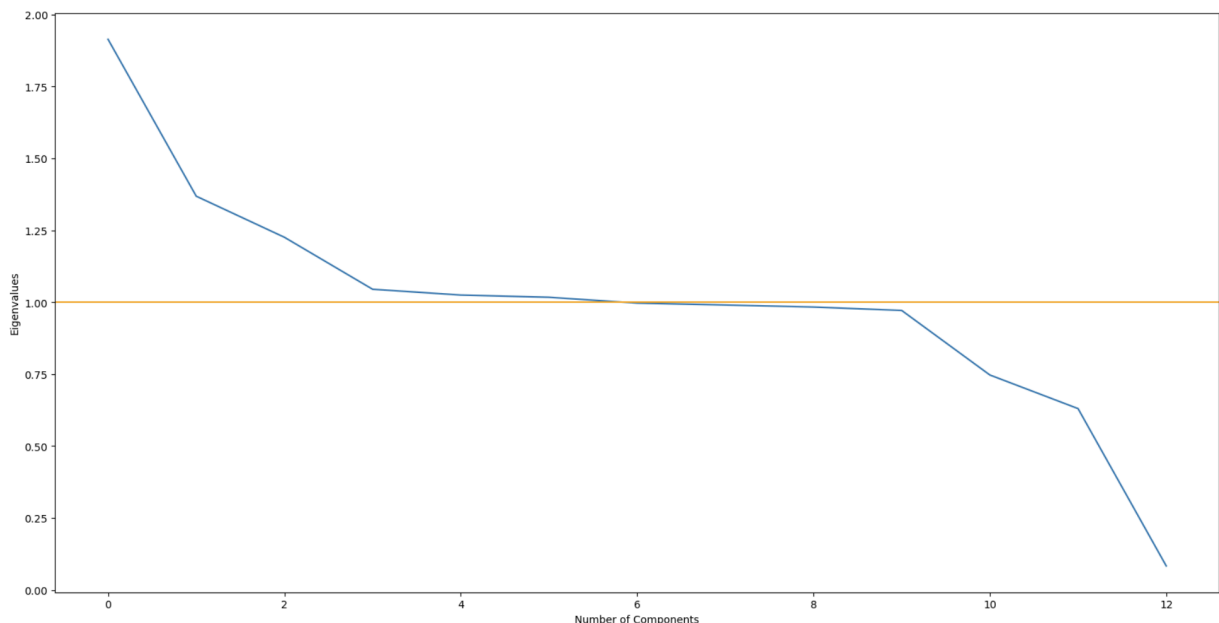
	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11	PC12
latitude_gps_coordinate	-0.019040	0.023955	-0.715140	0.117357	-0.060860	0.003342	-0.041279	-0.061812	-0.045629	-0.032326	0.679135	0.016400
longitude_gps_coordinate	-0.004369	0.028830	0.273628	-0.592108	-0.372867	0.340515	0.377267	0.090325	-0.090028	0.108452	0.383671	0.033852
population	0.024136	-0.056457	0.629736	0.288576	0.225043	-0.161601	-0.225558	-0.068005	0.035257	-0.030678	0.617999	-0.004212
children_in_household	-0.001480	0.008555	0.015820	0.241122	0.200363	-0.238385	0.792937	-0.399989	-0.191821	0.135710	-0.000267	-0.024660
age	0.027347	0.703845	0.033119	0.008760	-0.043299	-0.031647	-0.028121	0.004472	-0.024380	-0.040154	0.017715	-0.704576
policyholder_income	-0.002999	0.015667	0.020498	0.382457	0.131638	0.401748	0.358316	0.555339	0.107582	-0.474793	0.021429	0.008224
vitamin_d_level	0.562169	-0.001437	-0.044499	-0.193577	0.294934	0.236292	-0.008809	-0.306174	0.228786	-0.193534	0.007792	-0.016595
physician_visits	-0.007408	0.003876	0.012389	0.223005	0.198445	0.634883	-0.188456	-0.120668	-0.571329	0.356800	-0.054313	-0.028954
full_meals_consumed	-0.007822	0.036794	-0.101559	-0.225136	0.583100	-0.085281	0.086984	0.454708	0.237512	0.558935	0.073346	-0.041888
vitamin_d_supplements	0.032285	-0.001060	0.029638	0.393395	-0.375709	0.276738	0.054142	-0.155063	0.629205	0.448624	-0.017236	-0.033783
inital_days_hospitalized	0.423938	-0.061466	0.018333	0.228461	-0.373662	-0.308579	0.022840	0.419290	-0.328890	0.247346	-0.007135	-0.009715
avg_daily_charges	0.706772	-0.028468	-0.020367	-0.012992	0.005689	0.001962	0.004402	0.005816	-0.018495	-0.006370	-0.001213	0.005109
avg_additional_charges	0.042828	0.702660	0.035909	0.053455	0.009405	-0.026474	-0.022945	-0.006145	-0.004948	0.033766	-0.011685	0.705219

Principal components: PC1, PC2, PC3, PC4, PC5, PC6

E2: Justification of Reduced Number of Principal Components

The scree plot (shown below) reveals that the first five principal components (PCs), PC1, PC2, PC3, PC4, PC5, and PC6, possess eigenvalues exceeding 1, indicating they capture a significant portion of the data's variance. While the sixth PC's eigenvalue appears close to the threshold of 1 visually, consulting the numerical values by printing out our array of 'eigenvalues' confirms it is still slightly above 1. All 6 of these PCs would be retained for further analysis. We can also see PCs 7-13 exhibit eigenvalues below 1, suggesting minimal contribution to the overall variance and justifying their exclusion.

```
# Generate Scree Plot
plt.figure(figsize = [20,10])
plt.plot(eigenvalues)
plt.xlabel('Number of Components')
plt.ylabel('Eigenvalues')
plt.axhline(y=1, color='orange')
plt.show()
```



```
# Print array of eigenvalues
eigenvalues
```

```
[1.9136268444029696,
 1.3684900837103073,
 1.22574526908525,
 1.045100058263724,
 1.0249832616301577,
 1.0172706643103397,
 0.9969944209674451,
 0.9903851136834361,
 0.9833121363165764,
 0.9714989569432093,
 0.7471998134173358,
 0.6304251196221257,
 0.08366825764688132]
```

E3: How Our Organization Would Benefit From PCA

With healthcare research, datasets often contain many different variables, posing challenges for analysis and machine learning models. Principal Component Analysis (PCA), an unsupervised learning technique, offers a powerful solution by reducing dimensionality while preserving essential information needed to answer our research questions. The application of PCA in analyzing data related to hospital readmission, specifically focusing on variables like days spent hospitalized, average daily charges and average additional charges.

Addressing the Curse of Dimensionality

High-dimensional data often suffers from the “curse of dimensionality”, where performance deteriorates with increasing data dimensions (Yiu, 2019). PCA solves this issue by transforming such data into a lower-dimensional space, condensing information into a smaller set of uncorrelated features, termed ‘principal components’ (PCs). This enables efficient model training, faster computation, and reduced resource requirements without significant accuracy loss (Bigabid, n.d.).

Application Within Our Project

PCA typically requires quantitative variables, however our dataset presents a unique opportunity having limited quantitative data. With PCA we’re able to facilitate analysis of some interrelated variables, such as days spent hospitalized, charges, etc, by combining them into a single PC. This PC then serves as the basis for analysis, replacing individual tests and preserving the underlying relationships between the variables. This approach adapts to the specific data structure, offering a more efficient and informative analysis.

Additional Benefits

Beyond dimensionality reduction, PCA offers additional advantages for such uses as readmission prediction:

- **Noise Reduction:** By capturing the most significant patterns in the data, PCA can effectively filter out noise and improve the signal-to-noise ratio, leading to more robust models.
- **Feature Selection:** PCA can indirectly contribute to feature selection by identifying the most relevant components that explain the majority of the data variance. This helps eliminate redundant features, potentially improving model performance.
- **Data Visualization:** PCA can be used to visualize high-dimensional data in lower dimensions, providing valuable insights into the underlying structure and relationships between variables.

In conclusion, PCA's ability to reduce dimensionality while preserving key information offers a valuable tool for enhancing machine learning models. By addressing the "curse of dimensionality," PCA improves generalization, optimizes computational efficiency, reduces noise, aids in feature selection, and facilitates data visualization and clustering. These benefits make PCA a relevant and powerful technique for various machine learning applications.

Part IV: Supporting Documents

F: Video Demonstration

Panopto video recording:

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=3e5c2f11-5654-49f0-bbb3-b124011d68c3>

G: Data Source

Western Governors University. (2023). *Medical Data and Dictionary Files* [Dataset]. Data Files and Associated Dictionary Files.

Retrieved January 10, 2024, from
<https://web5.wgu.edu/aap/content/d206-ema.html>

H: References

Bigabid (n.d.). *PCA: What, How, and Why*.

Retrieved February 23, 2024, from
<https://www.bigabid.com/what-is-pca-and-how-can-i-use-it/>

Boeye, J. (n.d.). *Dimensionality Reduction in Python* [Interactive Course]. Datacamp.

Retrieved January 10, 2024, from
<https://app.datacamp.com/learn/courses/dimensionality-reduction-in-python>

Browe-Anderson, H. (n.d.). *Introduction to Python* [Interactive Course]. Datacamp.

Retrieved January 10, 2024, from
<https://app.datacamp.com/learn/courses/intro-to-python-for-data-science>

Browe-Anderson, H. (n.d.). *Introduction to Importing Data in Python* [Interactive Course]. Datacamp.

Retrieved January 10, 2024, from
<https://app.datacamp.com/learn/courses/introduction-to-importing-data-in-python>

Centers for Disease Control and Prevention (2023, September 5). *Diabetes Basics*.

Retrieved January 31, 2024, from
<https://www.cdc.gov/diabetes/basics/index.html>

Dixon, A. E. & Nyenhuis, S. M. (2022, May 17). Obesity and asthma. *UpToDate*.

Retrieved January 31, 2024, from
<https://www.uptodate.com/contents/obesity-and-asthma>

Donthi, S. (n.d.). *Dealing with Missing Data in Python* [Interactive Course]. Datacamp.

Retrieved January 10, 2024, from

<https://app.datacamp.com/learn/courses/dealing-with-missing-data-in-python>

Johns Hopkins Medicine (n.d.). *Metabolic Syndrome*.

Retrieved January 31, 2024, from

<https://www.hopkinsmedicine.org/health/conditions-and-diseases/metabolic-syndrome>

Larose, C., Larose, D. (2019). *Data Science Using Python and R*. Wiley.

Nehme, A. (n.d.). *Cleaning Data in Python* [Interactive Course]. Datacamp.

Retrieved January 10, 2024, from

<https://app.datacamp.com/learn/courses/cleaning-data-in-python>

Whelan, C. (2023, May 25). Can Obesity Cause Back Pain? *Healthline*.

Retrieved January 31, 2024, from

<https://www.healthline.com/health/obesity/back-pain-and-obesity>

Yiu, T. (2019, July 20). *The Curse of Dimensionality*. Towards Data Science.

Retrieved February 23, 2024, from

<https://towardsdatascience.com/the-curse-of-dimensionality-50dc6e49aa1e>